

BÀI 26: HÀM TRONG PYTHON

SAU BÀI NÀY EM SẼ

- Biết được chương trình con là hàm.
- Biết cách tạo hàm.

KHỞI ĐỘNG

Các chương trình phức tạp thường có những khối lệnh được lặp đi lặp lại ở nhiều vị trí khác nhau. Để tránh phải viết lại, người ta thường gom các khối lệnh đó thành những **chương trình con**. Trong Python, các hàm chính là các chương trình con. Khi cần, ta chỉ việc gọi tên hàm thay vì viết lại cả khối lệnh. Em có thể kể tên một số hàm đã học (như `print()`, `len()`, `int()`) và cho biết chúng có đặc điểm chung gì không?

1. MỘT SỐ HÀM THIẾT KẾ SẴN CỦA PYTHON

Hoạt động 1: Tìm hiểu một số hàm của Python

Python cung cấp sẵn nhiều hàm để thực hiện các công việc khác nhau. Dưới đây là một số hàm có sẵn:

<code>abs()</code>	<code>len()</code>	<code>range()</code>	<code>bool()</code>	<code>float()</code>
<code>list()</code>	<code>round()</code>	<code>chr()</code>	<code>input()</code>	<code>ord()</code>
<code>str()</code>	<code>divmod()</code>	<code>int()</code>	<code>print()</code>	<code>type()</code>

Điểm chung của các lệnh trên là chúng đều có **tên hàm** theo sau là cặp dấu ngoặc đơn `()`. Bên trong dấu ngoặc là các **tham số** (dữ liệu đầu vào cho hàm).

Ví dụ:

- Lệnh `print("Thời khoá biểu")` có tham số là chuỗi "Thời khoá biểu".
- Lệnh `x = int("52")` có tham số là chuỗi "52".
- Trong một số trường hợp, bên trong dấu ngoặc có thể bỏ trống, ví dụ `x = input()`.

Cú pháp chung để gọi hàm:

<tên hàm>(<danh sách tham số>)

GHI NHỚ

Python cung cấp sẵn nhiều hàm thực hiện những công việc khác nhau, cho phép người dùng tùy ý sử dụng khi viết chương trình bằng các câu lệnh gọi hàm tương ứng.

2. THIẾT LẬP CÁC HÀM TỰ ĐỊNH NGHĨA

Ngoài các hàm có sẵn, Python cho phép người dùng tự thiết lập các hàm của riêng mình.

Hoạt động 2: Cách thiết lập hàm trong Python

Hàm trong Python được định nghĩa bằng từ khoá `def`, theo sau là tên hàm và cặp dấu ngoặc đơn `()`. Khối lệnh của hàm được viết sau dấu `:` và lùi vào.

Ví dụ 1: Cách viết hàm có trả lại giá trị.

Hàm `inc(n)` nhận vào một số `n` và trả về giá trị `n + 1`.

Python

```
>>> def inc(n):      # Tên hàm: inc. Tham số: n.
...     return n + 1 # Lệnh return để trả về giá trị
...
>>> inc(3)
4
```

Ví dụ 2: Cách viết hàm không trả lại giá trị.

Hàm `thong_bao(msg)` nhận vào một xâu và in ra lời chào.

Python

```
>>> def thong_bao(msg): # Tên hàm: thong_bao. Tham số: msg.
...     print("Xin chào bạn", msg)
...     return          # return không có giá trị (có thể bỏ qua)
...
>>> thong_bao("Trần Quang Minh")
Xin chào bạn Trần Quang Minh
```

Cú pháp thiết lập hàm:

- **Hàm có trả lại giá trị:**

Python

```
def <tên hàm>(<tham số>):
    <khối lệnh>
    return <giá trị>
```

- **Hàm không trả lại giá trị:**

Python

```
def <tên hàm>(<tham số>):
    <khối lệnh>
    return
```

(Nếu hàm không trả lại giá trị, lệnh `return` có thể được bỏ qua).

GHI NHỚ

- Để thiết lập hàm trả lại giá trị, câu lệnh `return` cần có giá trị đi kèm.
- Để thiết lập hàm không trả lại giá trị, có thể dùng lệnh `return` không có giá trị hoặc không cần có `return`.

THỰC HÀNH

Nhiệm vụ 1: Viết hàm yêu cầu người dùng nhập họ tên rồi đưa lời chào ra màn hình.

Python

```
def meeting():
    ten = input("Nhập họ tên của em: ")
    print("Xin chào", ten)
```

Gọi hàm để thực thi

`meeting()`

Nhiệm vụ 2: Viết hàm `prime(n)` với tham số là số tự nhiên `n`. Hàm trả lại `True` nếu `n` là số nguyên tố, và `False` nếu ngược lại.

Hướng dẫn: Số nguyên tố là số tự nhiên lớn hơn 1 và chỉ có 2 ước là 1 và chính nó.

Python

```
def prime(n):
    if n < 2:
        return False
    C = 0 # Biến đếm số ước
    k = 1
    while k <= n:
        if n % k == 0:
            C = C + 1
            k = k + 1

    if C == 2:
        return True
    else:
        return False
```

Ví dụ sử dụng hàm

```
so = int(input("Nhập một số để kiểm tra: "))
if prime(so):
    print(so, "là số nguyên tố.")
else:
    print(so, "không phải là số nguyên tố.")
```

LUYỆN TẬP

- Viết hàm với tham số là số tự nhiên n , in ra các số là ước nguyên tố của n . (Gợi ý: Sử dụng lại hàm `prime()` ở trên).
- Viết hàm `numbers(s)` đếm số các chữ số có trong chuỗi s . Ví dụ `numbers("0101abc")` trả về 4.

VẬN DỤNG

- Trong khi viết hàm có thể có nhiều lệnh `return`. Quan sát hàm sau và giải thích ý nghĩa. Hàm này có điểm gì khác so với hàm `prime()` trong phần thực hành?

Python

```
def prime_v2(n):
    if n < 2:
        return False
    k = 2
    while k < n:
        if n % k == 0:
            return False # Thoát hàm ngay khi tìm thấy ước
        k = k + 1
    return True # Nếu vòng lặp kết thúc mà không tìm thấy ước
```

- Viết chương trình yêu cầu nhập một chuỗi kí tự, sau đó thông báo:
 - Tổng số các kí tự là chữ số của chuỗi.
 - Tổng số các kí tự là chữ cái tiếng Anh trong chuỗi.

(Yêu cầu: Viết hàm cho mỗi yêu cầu trên).